

BILAN ENTREES SORTIES Proc LPC1769 partie BASE						
NOM MODULE	NOM SIGNAL	ANA IN/ ANA OUT	PERIPHERIQUE	PIN FUNCTION	Px.y	Gestion par IT ou tâche / variables associées
encadré Schéma	Compatible Doc Elec	NUM IN / NUM OUT	NOM / NUMERO	id nom tab pinsel		
ECRAN DE SUPERVISION	BASE_TO_SUPER	NUM OUT	UART3	TXD3 (0b01) PINSELO	P0.0	Après l'acquisition du message "RXvS\r\n" ou "lettrePyy" de la poste, on ajoute "Pnn" en entête et on transmet le message ("PnnRXvS" ou "PnnlettrePyy") vers la supervision.
	SUPER_TO_BASE	NUM IN		RXD3 (0b01<<2) PINSELO	P0.1	La base reçoit des messages consigne vitesse de type "RXVyy\r\n" (X - ID robot, yy - vitesse, \r\n retour à la ligne qui indique le fin du message) et aussi des messages d'attribution de type "RXPss-Pdd\r\n" (X - ID robot, ss - numéro poste de départ, dd - numéro poste d'arrivé, \r\n retour à la ligne qui indique le fin du message). On va autoriser une interruption sur UART3, et la fonction UART3_IRQHandler() va gérer l'acquisition du message en le mettant dans une tableau buffer nommé rxBuffer[] jusqu'à atteindre le fin du message. On met à jour les tableaux : 1. si le message est de la forme "RXVyy\r\n" vitesse_voulue[X] = (rxBuffer[3]- '0')*10 + (rxBuffer[4]- '0'); (X - ID robot, 3 et 4 - position de la vitesse dans le message reçu). 2. si le message est de la forme "RXPss-Pdd\r\n" enlevement[X] = [rxBuffer[3];rxBuffer[4]]; et livraison[X] = [rxBuffer[7];rxBuffer[8]];
POSTES DE TRAVAIL	BASE_TO_POSTE	NUM OUT	UART0	TXD0 (0b01<<4) PINSELO	P0.2	La base envoie un message de type "@Tnn\r\n" (nn - numéro de robot, \r\n retour à la ligne qui indique le fin du message). On va créer une fonction UART0_envoye_char(char c); qui va envoyer un caractère si UART0->LSR[5] == 1, ça veut dire qu'on envoie si le registre de transmission est vide.
	POSTE_TO_BASE	NUM IN		RXD0 (0b01<<6) PINSELO	P0.3	La base reçoit des messages type "RXvS\r\n" (X - ID robot, v - vitesse, S - status, \r\n retour à la ligne qui indique le fin du message). On va autoriser une interruption sur UART0, et la fonction UART0_IRQHandler() va gérer l'acquisition du message en le mettant dans une tableau de type buffer nommé rxBuffer[] jusqu'à atteindre le fin du message. On met à jour les tableaux : 1. vitesse_actuelle[X] = rxBuffer[2] (X - ID robot, et 2 - position de la vitesse dans le message reçu); 2. on vérifie si le robot à change de status et on mis à jour les tableaux enlevement[] et livraison[]. Si status changé à enlevecolis, enlevement[X] = "00". Si status changé à livraison, livraison[X] = "00".
DIP SWITCH NOMBRE DE ROBOTS	NB_ROB[0]	NUM IN	GPIO0	GPIO Port 0.4 (0b00<<8) PINSELO	P0.4	On va autoriser les interruptions sur front montant et front descendant. Dans l'EINT3_Handler() on va mettre à jour la variable globale NB_ROBOTS à chaque changement d'état des DIP SWITCHs.
	NB_ROB[1]			GPIO Port 0.5 (0b00<<10) PINSELO	P0.5	
	NB_ROB[2]			GPIO Port 0.6 (0b00<<12) PINSELO	P0.6	
	NB_ROB[3]			GPIO Port 0.7 (0b00<<14) PINSELO	P0.7	
DIP SWITCH NOMBRE DE POSTES	NB_POSTE[0]	NUM IN	GPIO0	GPIO Port 0.8 (0b00<<16) PINSELO	P0.8	On va autoriser les interruptions sur front montant et front descendant. Dans l'EINT3_IRQHandler() on va mettre à jour la variable globale NB_POSTES à chaque changement d'état des DIP SWITCHs.
	NB_POSTE[1]			GPIO Port 0.9 (0b00<<18) PINSELO	P0.9	
	NB_POSTE[2]			GPIO Port 0.10 (0b00<<20) PINSELO	P0.10	
	NB_POSTE[3]			GPIO Port 0.11 (0b00<<22) PINSELO	P0.11	
FIL CONDUCTEUR	COURANT MODULE	NUM OUT	PWM1	PWM1.1 (0b10<<4) PINSEL3	P1.18	On va générer un signal PWM avec une fréquence de 50 kHz. On utilisera un Timer pour mesurer la durée pendant laquelle le PWM est actif (émission) ou inactif (pause). Pour envoyer l'entête ('E'), le PWM sera actif pendant 2,5 ms, puis désactivé pendant 0,5 ms. Pour envoyer un '0' logique, le PWM sera actif pendant 1,0 ms, puis désactivé pendant 0,5 ms. Pour envoyer un '1' logique, le PWM sera actif pendant 1,75 ms, puis désactivé pendant 0,5 ms.